# Feature Store: Manage Features and Generate Training Data

The Snowflake Feature Store provides a centralised repository for feature engineering pipelines, enabling consistent feature reuse across training and inference. `snowflakeR` wraps the full Feature Store Python API, giving you an R-native interface.

## Concepts

- **Entity**: Defines join keys for features (e.g., `CUSTOMER_ID`). Immutable once created.
- **Feature View**: A set of features derived from a SQL query, tied to one or more entities. Can be automatically refreshed (managed) or manually maintained (external).
- **Feature Store**: A schema in Snowflake that holds entities, feature views, and metadata.

## Setup

```r
library(snowflakeR)

conn <- sfr_connect()

# Connect to (or create) a Feature Store
fs <- sfr_feature_store(
  conn,
  database  = "ML_DB",
  schema    = "FEATURES",
  warehouse = "ML_WH",
  create    = TRUE  # Creates schema + tags if they don't exist
)

fs
#> <sfr_feature_store> "ML_DB"."FEATURES"
#>   warehouse: "ML_WH"
#>   mode: "create"
```

### One-time admin setup

For first-time setup with proper role/privilege provisioning:

```r
sfr_setup_feature_store(conn, "ML_DB", "FEATURES", "ML_WH")
```

## Entities

Entities define the join keys that link features to business objects:

```r
# Create entities
customer <- sfr_create_entity(fs, "CUSTOMER", "CUSTOMER_ID", desc = "Customer entity")
product  <- sfr_create_entity(fs, "PRODUCT", "PRODUCT_ID", desc = "Product entity")

customer
#> <sfr_entity> "CUSTOMER"
```

```r
#>   join_keys: "CUSTOMER_ID"

# List all entities
sfr_list_entities(fs)

# Get a specific entity
customer <- sfr_get_entity(fs, "CUSTOMER")

# Update description
sfr_update_entity(fs, "CUSTOMER", desc = "Primary customer entity with demographics")

# Delete an entity (fails if referenced by Feature Views)
sfr_delete_entity(fs, "PRODUCT")
```

## Feature Views

Feature Views define the SQL transformation that produces features. There are two approaches.

### Two-step: draft then register

This mirrors the Python API exactly and is useful when you want to inspect the draft before materialising:

```r
# Step 1: Create a local draft
fv_draft <- sfr_feature_view(
  name         = "CUSTOMER_FEATURES",
  entities     = customer,
  feature_df   = "
    SELECT
      customer_id,
      AVG(order_total) AS avg_order_total,
      COUNT(*) AS order_count,
      MAX(order_date) AS last_order_date
    FROM orders
    GROUP BY customer_id
  ",
  refresh_freq = "1 hour",
  desc         = "Customer aggregate features from orders"
)

fv_draft
#> <sfr_feature_view> "CUSTOMER_FEATURES" [draft]
#>   entities: "CUSTOMER"
#>   refresh: "1 hour"

# Step 2: Register (materialise as a dynamic table)
fv <- sfr_register_feature_view(fs, fv_draft, version = "v1")
#> Feature View "CUSTOMER_FEATURES" version "v1" registered.
```

### One-step convenience

```r
fv <- sfr_create_feature_view(
  fs,
  name         = "CUSTOMER_FEATURES",
  version      = "v1",
```

```
  entities     = customer,
  feature_df   = "
    SELECT customer_id, AVG(order_total) AS avg_order_total,
           COUNT(*) AS order_count
    FROM orders GROUP BY customer_id
  ",
  refresh_freq = "1 hour",
  desc         = "Customer order aggregates"
)
```

### Using dbplyr for feature_df

You can define feature transformations using dplyr pipelines instead of raw SQL. The `feature_df` argument accepts dbplyr lazy tables. This requires the RSnowflake package for DBI/dbplyr connectivity:

```
library(dplyr)
library(dbplyr)

# Get a DBI connection from the sfr_connection
dbi_con <- sfr_dbi_connection(conn)

orders_tbl <- tbl(dbi_con, "ORDERS")

customer_features_query <- orders_tbl |>
  group_by(customer_id) |>
  summarise(
    avg_order_total = mean(order_total, na.rm = TRUE),
    order_count = n(),
    last_order_date = max(order_date, na.rm = TRUE)
  )

fv <- sfr_create_feature_view(
  fs,
  name     = "CUSTOMER_FEATURES_DBPLYR",
  version  = "v1",
  entities = customer,
  feature_df = customer_features_query,  # dbplyr lazy table -> SQL
  desc       = "Customer features built with dplyr"
)
```

### External (manually managed) Feature Views

Omit `refresh_freq` to create a Feature View without automatic refresh. You maintain the underlying table yourself (e.g., via dbt):

```
fv_external <- sfr_create_feature_view(
  fs,
  name     = "CUSTOMER_DEMOGRAPHICS",
  version  = "v1",
  entities = customer,
  feature_df = "SELECT * FROM customer_demographics_table"
  # No refresh_freq -> external Feature View
)
```

### Time-series features

For point-in-time correct joins, specify a `timestamp_col`:

```
fv_ts <- sfr_create_feature_view(
  fs,
  name          = "CUSTOMER_ACTIVITY",
  version       = "v1",
  entities      = customer,
  feature_df    = "SELECT customer_id, event_ts, page_views, clicks FROM activity",
  timestamp_col = "event_ts",
  refresh_freq  = "30 minutes"
)
```

## Managing Feature Views

```
# List all Feature Views
sfr_list_feature_views(fs)

# Filter by entity or name
sfr_list_feature_views(fs, entity_name = "CUSTOMER")
sfr_list_feature_views(fs, feature_view_name = "CUSTOMER_FEATURES")

# Get a specific version
fv <- sfr_get_feature_view(fs, "CUSTOMER_FEATURES", "v1")

# Show all versions of a Feature View
sfr_show_fv_versions(fs, "CUSTOMER_FEATURES")

# Read feature data
feature_data <- sfr_read_feature_view(fs, "CUSTOMER_FEATURES", "v1")
head(feature_data)
```

### Refresh management

```
# Manually trigger a refresh
sfr_refresh_feature_view(fs, "CUSTOMER_FEATURES", "v1")

# Check refresh history
history <- sfr_get_refresh_history(fs, "CUSTOMER_FEATURES", "v1")
history
#>              name    state      refresh_start_time ...
#> 1 CUSTOMER_FE...  SUCCEEDED 2026-02-10 14:53:58  ...

# Pause automatic refresh
sfr_suspend_feature_view(fs, "CUSTOMER_FEATURES", "v1")

# Resume automatic refresh
sfr_resume_feature_view(fs, "CUSTOMER_FEATURES", "v1")
```

## Generate Training Data

Join spine (label) data with features using point-in-time correct joins:

```r
training_data <- sfr_generate_training_data(
  fs,
  spine = "
    SELECT customer_id, churn_date AS event_ts, churned AS label
    FROM churn_labels
  ",
  features = list(
    list(name = "CUSTOMER_FEATURES", version = "v1"),
    list(name = "CUSTOMER_ACTIVITY", version = "v1")
  ),
  spine_timestamp_col = "event_ts",
  spine_label_cols    = "label"
)

head(training_data)
#>   customer_id event_ts label avg_order_total order_count page_views clicks
#> 1       C001  2026-... 1          45.20          12         320     45
#> ...
```

You can also pass registered `sfr_feature_view` objects directly:

```r
fv1 <- sfr_get_feature_view(fs, "CUSTOMER_FEATURES", "v1")
fv2 <- sfr_get_feature_view(fs, "CUSTOMER_ACTIVITY", "v1")

training_data <- sfr_generate_training_data(
  fs,
  spine    = "SELECT customer_id, churned FROM churn_labels",
  features = list(fv1, fv2),
  spine_label_cols = "churned"
)
```

#### Materialise training data to a table

```r
training_data <- sfr_generate_training_data(
  fs,
  spine    = "SELECT customer_id, churned FROM churn_labels",
  features = list(fv1, fv2),
  save_as  = "CHURN_TRAINING_SET_V1"
)
```

### Retrieve Features for Inference

At inference time, fetch the latest feature values without labels or point-in-time logic:

```r
features_for_scoring <- sfr_retrieve_features(
  fs,
  spine    = "SELECT customer_id FROM customers_to_score",
  features = list(fv1, fv2)
)
```

### Tile-based aggregation

For rolling-window features, define aggregation features with `sfr_feature()` and set a `feature_granularity`:

```r
feats <- list(
  sfr_feature("AMOUNT", "FLOAT", "SUM", "1 hour"),
  sfr_feature("AMOUNT", "FLOAT", "AVG", "1 day")
)

fv_agg <- sfr_create_feature_view(
  fs,
  name        = "TXN_AGGREGATES",
  version     = "v1",
  entities    = customer,
  feature_df  = "SELECT customer_id, event_ts, amount FROM transactions",
  features    = feats,
  feature_granularity = "1 DAY",
  timestamp_col = "event_ts",
  refresh_freq  = "1 hour"
)
```

## Online serving

Enable low-latency online feature lookups by attaching an `sfr_online_config()` to a Feature View:

```r
oc <- sfr_online_config(enable = TRUE, target_lag = "2 minutes")

fv_online <- sfr_create_feature_view(
  fs,
  name          = "CUSTOMER_ONLINE",
  version       = "v1",
  entities      = customer,
  feature_df    = "SELECT customer_id, credit_score FROM credit_data",
  refresh_freq  = "5 minutes",
  online_config = oc
)

# Read from the online store
online_data <- sfr_read_feature_view(
  fs, "CUSTOMER_ONLINE", "v1",
  store_type = "ONLINE"
)

# Refresh and check history for online store
sfr_refresh_feature_view(fs, "CUSTOMER_ONLINE", "v1", store_type = "ONLINE")
history <- sfr_get_refresh_history(fs, "CUSTOMER_ONLINE", "v1", store_type = "ONLINE")
```

You can also enable online serving on an existing Feature View:

```r
sfr_update_feature_view(
  fs, "CUSTOMER_FEATURES", "v1",
  online_config = sfr_online_config(enable = TRUE, target_lag = "5 minutes")
)
```

## Advanced Feature View options

The `sfr_feature_view()` and `sfr_create_feature_view()` constructors accept several additional parameters:

```r
fv <- sfr_create_feature_view(
  fs,
  name         = "CLUSTERED_FV",
  version      = "v1",
  entities     = customer,
  feature_df   = "SELECT customer_id, amount FROM orders",
  refresh_freq = "1 hour",
  initialize   = "ON_CREATE",    # populate immediately on creation
  refresh_mode = "INCREMENTAL",  # incremental refresh (vs full)
  cluster_by   = c("CUSTOMER_ID"),  # cluster the backing table
  block        = TRUE            # wait for registration to complete
)
```

| Parameter | Description |
|---|---|
| `initialize` | `"ON_CREATE"` populates immediately; `NULL` defers first refresh |
| `refresh_mode` | `"INCREMENTAL"` or `"FULL"` |
| `cluster_by` | Character vector of columns to cluster the dynamic table |
| `block` | If `TRUE` (default), `sfr_register_feature_view()` waits for registration |

## Feature View introspection

Inspect a registered Feature View's metadata without reading its data:

```r
# List columns and their types
cols <- sfr_list_fv_columns(fs, "CUSTOMER_FEATURES", "v1")
cols
#>     NAME CATEGORY  DTYPE DESC
#> 1 CUST_ID  ENTITY STRING
#> 2  AMOUNT FEATURE  FLOAT  Transaction amount

# Get the underlying SQL query
sfr_fv_query(fs, "CUSTOMER_FEATURES", "v1")
#> "SELECT customer_id, amount FROM orders"

# Get the fully qualified name (useful for SQL references)
sfr_fv_fqn(fs, "CUSTOMER_FEATURES", "v1")
#> "ML_DB.FEATURES.CUSTOMER_FEATURES$v1"

# Read data as a data.frame
df <- sfr_fv_to_df(fs, "CUSTOMER_FEATURES", "v1")
```

## Feature subsetting and descriptions

Attach human-readable descriptions to individual features, and create sliced views that expose only selected columns:

```r
# Attach per-feature descriptions
sfr_attach_feature_desc(
  fs, "CUSTOMER_FEATURES", "v1",
  descs = c(
    avg_order_total = "Average order amount in USD",
```

```r
    order_count     = "Total number of orders"
  )
)

# Slice to a subset of features
sliced_fv <- sfr_slice_feature_view(
  fs, "CUSTOMER_FEATURES", "v1",
  feature_names = c("avg_order_total")
)
```

Sliced views can be passed to `sfr_generate_training_data()` and `sfr_retrieve_features()` just like full Feature Views.

## Advanced training data parameters

The training data generation and retrieval functions accept additional parameters for controlling joins and output:

```r
# Exclude columns from the result
training <- sfr_generate_training_data(
  fs,
  spine    = "SELECT customer_id, churned FROM labels",
  features = list(fv1, fv2),
  exclude_columns = c("last_order_date"),  # drop from output
  auto_prefix = TRUE,  # prefix columns with FV name to avoid collisions
  include_feature_view_timestamp_col = TRUE,  # include FV timestamps
  join_method = "AS_OF"  # explicit join method
)

# Same parameters work on sfr_generate_dataset and sfr_retrieve_features
ds <- sfr_generate_dataset(
  fs, "TRAINING_V2", spine = "...", features = list(fv1),
  exclude_columns = c("internal_col"),
  output_type = "TABLE"  # materialise as a table (vs default)
)
```

| Parameter | Available on | Description |
|---|---|---|
| `exclude_columns` | training, dataset, retrieve | Columns to drop from output |
| `auto_prefix` | training, dataset, retrieve | Prefix columns with FV name |
| `include_feature_view_timestamp_col` | training, dataset, retrieve | Include FV timestamp columns |
| `join_method` | training, dataset, retrieve | Join strategy (`"AS_OF"`, etc.) |
| `output_type` | dataset only | Output format (`"TABLE"`, etc.) |

## Lineage and Dataset utilities

Trace upstream and downstream lineage from a Feature View, and recover which Feature Views produced a given Dataset:

```r
# Feature View lineage
lineage <- sfr_fv_lineage(fs, "CUSTOMER_FEATURES", "v1",
                          direction = "both")

# Load Feature Views associated with a Dataset
fvs <- sfr_load_fvs_from_dataset(fs, "CHURN_TRAINING", "v1")
```

```r
fvs
#> [[1]]
#> $name: "CUSTOMER_FEATURES"
#> $version: "v1"
```

### Iceberg-backed Feature Views

For Feature Views backed by Apache Iceberg tables, create a storage configuration and pass the external volume to the Feature Store:

```r
# Create a Feature Store with an Iceberg external volume
fs_iceberg <- sfr_feature_store(
  conn,
  database = "ML_DB",
  schema   = "FEATURES",
  create   = TRUE,
  default_iceberg_external_volume = "my_iceberg_volume"
)

# Or create a StorageConfig object (requires snowflake-ml-python >= 1.26.0)
sc <- sfr_storage_config(
  format          = "ICEBERG",
  external_volume = "my_iceberg_volume",
  base_location   = "/features"
)
```

### Warehouse management

Change the default warehouse used by the Feature Store:

```r
sfr_update_default_warehouse(fs, "ML_WH_LARGE")
```

### Clean up

```r
sfr_delete_feature_view(fs, "CUSTOMER_FEATURES", "v1")
sfr_delete_feature_view(fs, "CUSTOMER_ACTIVITY", "v1")
sfr_delete_entity(fs, "CUSTOMER")
```

### End-to-end workflow

Here's a complete example tying everything together:

```r
library(snowflakeR)

# 1. Connect
conn <- sfr_connect()
fs   <- sfr_feature_store(conn, database = "ML_DB", schema = "FEATURES",
                          warehouse = "ML_WH", create = TRUE)
reg  <- sfr_model_registry(conn, database = "ML_DB", schema = "MODELS")

# 2. Create entity + feature view
entity <- sfr_create_entity(fs, "CUSTOMER", "CUSTOMER_ID")
fv <- sfr_create_feature_view(
  fs, "CUST_FV", "v1", entities = entity,
```

```r
  feature_df = "SELECT customer_id, avg_spend, tenure FROM customer_summary",
  refresh_freq = "1 hour"
)

# 3. Generate training data
training <- sfr_generate_training_data(
  fs,
  spine = "SELECT customer_id, churned FROM labels",
  features = list(list(name = "CUST_FV", version = "v1")),
  spine_label_cols = "churned"
)

# 4. Train in R
model <- glm(churned ~ avg_spend + tenure, data = training, family = binomial)

# 5. Test locally
sfr_predict_local(model, training[1:5, c("avg_spend", "tenure")])

# 6. Register
sfr_log_model(
  reg, model, "CHURN_MODEL",
  input_cols  = list(avg_spend = "double", tenure = "double"),
  output_cols = list(prediction = "double")
)

# 7. Score new customers
new_features <- sfr_retrieve_features(
  fs,
  spine = "SELECT customer_id FROM active_customers",
  features = list(list(name = "CUST_FV", version = "v1"))
)
predictions <- sfr_predict(reg, "CHURN_MODEL", new_features)
```